MASTER PROMPT: BUILD ANILOV (TERMUX CLI VERSION)

For Antigravity CLI (`agy`) running in Termux on an Android phone. Flutter app, built into an APK by GitHub Actions.

1. YOUR ROLE AND HOW TO WORK

Act as a senior Flutter engineer, mobile architect, UI designer, and QA engineer. Build the app **Anilov** in small, verified stages called RUNS (Section 11).

- Do only the RUN I name in my message. Do not start the next run on your own.
- Implement, commit, push, check CI, fix. Do not stop at plans or mockups.
- My AI usage quota is limited. Keep each run focused, do not rewrite files without reason, do not print large files back to me, and commit often so progress is saved if the quota runs out.

2. ENVIRONMENT (IMPORTANT)

- I work on an **Android phone in Termux**. There is no PC.
- Flutter, Dart, Java, and the Android SDK are **not installed locally and must not be installed**. Never run `flutter` , `dart` , or `gradle` locally.
- Local tools available: `git` , `gh` (GitHub CLI, already logged in), `python` , `node` , `curl` .
- All formatting, analysis, tests, and builds run in **GitHub Actions**. After every push: run `gh run watch` . If it fails, run `gh run view --log-failed` , fix the cause, and push again. After 3 failed fix attempts on the same problem, stop and explain it to me.
- Because Flutter is not available locally, write careful, conservative code that uses well-known APIs. **Do not use code generation** (no build_runner, freezed, json_serializable, drift, riverpod_generator). Parse JSON by hand with null-safe `fromJson` factories.
- To choose package versions, query `https://pub.dev/api/packages/<name>` with `curl` , use the latest stable version with a caret constraint, and adjust if CI reports a resolution conflict.
- Ask me before installing anything in Termux.
- Never print tokens. Never commit secrets, `.env` files, or keys.

3. IDENTITY AND LOGO

- **Name:** Anilov
- **Tagline:** Discover anime. Track every episode.
- **Platform:** Android
- **Application ID:** `com.anilov.anilov` (from `flutter create --org com.anilov --project-name anilov`)
- **Final deliverable:** A debug APK that I can download from GitHub Actions and install on my phone.
- **Style:** Dark charcoal background, deep indigo and violet accents, subtle gradients, rounded poster cards, clean typography, smooth transitions. Also support a light theme. Accessible contrast and touch targets of at least 48dp.

Logo (generate it with code, no external assets)

- Create `tools/make_icon.py` using Pillow. It draws a 1024x1024 rounded-square icon with a diagonal gradient from deep indigo (`#1B1464`) to violet (`#7C3AED`) and a centered white stylized play-triangle with a small sparkle. It also writes a transparent foreground-only version for the adaptive icon.
- Outputs: `assets/icon/icon.png` and `assets/icon/icon_foreground.png`.
- Use `flutter_launcher_icons` (dev dependency) with an adaptive icon, background color `#1B1464`.
- Splash: use `flutter_native_splash` with the same background color and the foreground logo, plus an in-app animated splash with the wordmark 'Anilov'. If the native splash step fails in CI, remove it and keep the in-app splash.
- Create a reusable `AnilovLogo` widget used in the app bar, onboarding, and About screen.
- Add 2 to 3 short onboarding screens.
- The workflow generates the icons before every build.

4. LEGAL BOUNDARIES (NON-NEGOTIABLE)

- Video playback and sources must be **authorized only**: official trailers (YouTube embed), official 'Where to watch' links returned by AniList, my own local video files, public-domain or sample videos that I explicitly provide, and future licensed providers.

- Never scrape websites. Never integrate unlicensed streaming sites, including any site whose link I or anyone else pastes. Never bypass DRM or authentication. Do not build an Aniyomi-style extension system.
- If I paste a link to a site that looks unlicensed, refuse that integration and offer a legal alternative.
- No fake data: no fake ratings, reviews, availability, episode IDs, or streaming URLs.
- Exclude adult content by default (`isAdult: false`).
- Show required attribution in the About screen: AniList, and ani.zip if it is used.
- Do not build a fake player that pretends to play real anime.

5. REFERENCE PROJECT

Reference: https://github.com/ebrahimHakimuddin/aniview. Use it for feature inspiration only. You may read its README (for example with `curl` or `gh repo view`). Do not copy its code, assets, logos, or branding, and do not clone it into this repository. If it is unavailable, continue without it.

6. TECHNOLOGY STACK

Flutter project at the **repository root** (not in a subfolder).

- Flutter stable, Dart, Material 3
- `flutter_riverpod` (no code generation)
- `go_router`
- `dio`
- `cached_network_image`
- `sqflite` and `path` for watchlist, progress, history, and cached metadata
- `shared_preferences` for settings
- `url_launcher` for official streaming links
- `youtube_player_iframe` for official trailers
- `video_player` and `chewie` for local playback (Run 5)
- `flutter_local_notifications` and `workmanager` for daily updates (Run 5)
- `mocktail` for tests
- Dev: `flutter_launcher_icons`, `flutter_native_splash`

No backend in this version.

7. CI BUILD PIPELINE

Create `.github/workflows/build-apk.yml`.

Triggers: push to `main`, and `workflow_dispatch`.

Steps (ubuntu-latest):

1. Check out the repository.
2. Set up Java 17 (Temurin) and stable Flutter with `subosito/flutter-action` (with caching).
3. If `android/` is missing: run `flutter create --platforms=android --org com.anilov --project-name anilov` in a **temporary directory** and copy only its `android/` folder into the repository. Never let it overwrite `lib/`, `pubspec.yaml`, or `test/`.
4. `python3 -m pip install pillow`, then `python3 tools/make_icon.py`.
5. `python3 tools/patch_android.py`. This script must be **idempotent** and must: set the app label to 'Anilov', add the `INTERNET` permission, add the `POST_NOTIFICATIONS` permission, and keep cleartext traffic disabled.
6. `flutter pub get`.
7. `dart run flutter_launcher_icons` and `dart run flutter_native_splash:create`.
8. `dart format .` (apply formatting in the runner so it never fails the build on style alone).
9. `flutter analyze`.
10. `flutter test`.
11. `flutter build apk --debug`.
12. Print the APK size and upload `build/app/outputs/flutter-apk/app-debug.apk` as an artifact named `anilov-debug-apk`.

Do not make `analyze` or `test` non-blocking to hide errors. Fix them.

If a plugin needs extra Android configuration (for example core library desugaring for notifications), add that in `tools/patch_android.py` and explain it.

8. FEATURES

Navigation

Bottom bar with five tabs: **Home, Explore, Schedule, Watchlist, Profile**. Profile contains Watch History, Settings, and About/Attribution. Preserve tab state and support Android back navigation.

Metadata provider

- Primary provider: **AniList GraphQL** (`https://graphql.anilist.co`, public data needs no key). Check its current official docs for rate limits, terms, and attribution.
- Put it behind a provider abstraction. Keep the API client separate from widgets.
- Handle nulls, timeouts, HTTP errors, and rate limits (HTTP 429, respect `Retry-After`). Cache suitable responses and avoid duplicate requests.
- If the API is unavailable, show a retry state. Never show fabricated data.
- Provide JSON fixtures for tests.

Home

Featured carousel, Airing today, Trending, Popular, Top rated, Currently airing, Continue watching, Recently viewed, genre shortcuts, search entry point. Use real data only. Include skeleton loaders, retry buttons, and empty states. A poster tap opens Details.

Schedule (core feature)

- Use AniList airing schedules for **today** and a **weekly view** with one tab per weekday.
- Convert times to the device's local time zone. Show episode number, time, cover, and title.
- Filter: All, or My watchlist only.
- Label times as original broadcast times. They do not imply availability on any service or region.
- Daily notification (Run 5): the user chooses a time in Settings. Once a day, summarize watchlist shows airing that day. Use `workmanager` and local notifications. Request `POST_NOTIFICATIONS` at runtime on Android 13+ with an explanation, and degrade gracefully if denied. Document that background timing is not exact. Allow a per-show reminder toggle.

Explore and search

Debounced title search, pagination, filters (genre, format, status, year), sort (popularity, rating, newest), search history with a clear button, no-results and error states.

Anime details

Cover and banner, titles and alternates, synopsis, genres, year, format, episode count, status, studio, average score with source, related anime if supported, **trailer** (when provided), **Where to watch** buttons, watchlist controls, watched-episode counter, continue action, and an episode list when reliable episode metadata exists. Keep episode metadata clearly separate from playable sources.

Optional episode enrichment: ani.zip

- Endpoint: `https://api.ani.zip/mappings?anilist_id=<id>`.
- Use it only for episode titles, thumbnails, air dates, and runtime, behind the provider abstraction. AniList stays the primary source.
- Never use it for video or streaming links.
- It is an unofficial community service. Parse every field as nullable, cache responses, and if it fails, times out, or returns nothing, fall back to plain episode numbers. The app must never crash or show fake data.
- Add a mocked-response unit test and an attribution line in About.

Watchlist and progress

Statuses: Planning, Watching, Completed, On hold, Dropped. Add and remove (removal needs confirmation), change status, update watched-episode count and last-watched episode, filter and sort. Persist locally with sqflite so data survives restarts. Prevent duplicates. Handle unknown episode totals and invalid values safely. Everything works without any account.

Watching (authorized sources)

Create a `PlaybackSource` abstraction with these sources:

1. Official 'Where to watch' links from AniList data, opened with `url_launcher`. Show only links the API returns. Never construct or guess links.
2. Official trailers via `youtube_player_iframe`, following the platform's terms.

3. Local media (Run 5): the user picks their own video files. Support play, pause, seek, full-screen, resume position, buffering indicator, and error handling.
4. A clear empty state when no licensed provider is configured.

Accounts (optional, Run 6)

Local-first. If added, use the official AniList OAuth flow with a custom-scheme redirect (for example `anilov://auth`), store tokens in `flutter_secure_storage`, and never create a screen that asks for a password. Ask me for the AniList client ID. Do not create external accounts yourself.

9. OFFLINE AND PERFORMANCE

The app opens and shows saved watchlist, progress, and history with no network. Cache images and reasonable metadata with expiry. Use efficient lists and loading skeletons. Dispose controllers and subscriptions. Log errors without leaking tokens.

10. TESTING

Write meaningful tests and let CI run them: model parsing with null and missing fields, mocked API client behavior, search and error states, schedule time-zone conversion and day grouping, watchlist persistence, duplicate prevention, progress updates, unknown totals, and key widget and navigation tests.

Never claim a test, analysis, or build passed unless the GitHub Actions run shows it passed.

11. RUN PLAN

Do only the run I name. At the end of each run: update `PROGRESS.md`, commit, push, check CI, and report **Completed / Tested in CI / Blocked / Not yet implemented**. Then stop.

- **RUN 1 — Foundation:** `.gitignore`, `pubspec.yaml`, theme, go_router with the 5-tab shell and placeholder screens, logo script, splash, onboarding, CI workflow, `tools/patch_android.py`, README, `PROGRESS.md`. Done when CI is green and the APK artifact exists.
- **RUN 2 — Data, Home, Explore, Details:** AniList client, models, provider abstraction, Home, Explore with search and filters, Details, with fixtures and tests.

- **RUN 3 — Persistence:** sqflite watchlist, progress, history, Continue watching, Watchlist tab, Profile history. Test restart and offline behavior.
- **RUN 4 — Schedule and sources:** Schedule tab, trailers, Where to watch buttons, ani.zip episode enrichment.
- **RUN 5 — Daily updates and polish:** notifications, settings, local-media playback, light theme polish, About and attribution.
- **RUN 6 — Optional:** AniList login and sync, release signing documentation.

12. RULES

1. Inspect the repository before creating or overwriting files. Never delete working code without my approval.
2. Work in small verified steps. If a command or CI job fails, read the error and fix its cause.
3. Never pretend something succeeded.
4. No nonfunctional buttons. Do not mark unfinished features as complete.
5. Never hardcode secrets or commit keys, tokens, or `.env` files.
6. Use official documentation for configuration.
7. Ask before installing anything, creating external accounts, using paid services, or publishing.
8. Do not violate any service's terms of use. Do not link to or integrate unauthorized copyrighted content.
9. The debug APK is signed with the debug key. Do not claim release signing is done unless it was configured and verified.

13. FIRST ACTION

Read this file. Run `git status`, `gh auth status`, and `ls` to confirm the environment, and report briefly. Then do the RUN that my message names. If my message names no run, wait for it.